LAB REPORT

MINI-RAG & LOCAL AI WITH OLLAMA

Prepared By:

Imane Samir

Contents

I. Introduction: 2
II. Experimental environment: 2
III. Experiments: 4
1. First model and first prompts: 4
2. Prompt Engineering: 4
3. Model Management: 5
4. Comparing two Models on a Coding Task: 6
5. Using Ollama from Python: 8
6. Chatbot Memory: 8
7. Local AI – Code Assistant 9
8. Document Assistant- Mini Rag: 10
9. Local AI Vision- Multimodal Model 11
10. Local AI Web Application: 12
11. Benchmark 13
IV. Final Project: FSTT Local AI Assistant 14
1. Architecture: 14
2. Results: 15
V. Conclusion 17

I. Introduction:

The report describes the Local AI lab using Ollama. The objective was to operate large language models (LLMs) entirely on my own machine and to investigate what this allows for: setting up and operating models, prompting, conducting experiments on models, invoking a model via the Python REST API, adding conversation memory, structured output and RAG, analyzing an image, and finally bringing all of these together in a web app, the FSTT Local AI Assistant.

All the figures have been extracted from the output of the notebook and the screenshots from the lab and are listed as figures. Timings depend on the specific hardware as well as on whether the model was loaded into memory first or not, thus tokens per second will be used as the speed measure.

II. Experimental environment:

The experiments have been performed on my personal computer with Windows 11 OS, where the Python programming language and Ollama were used to run large language models on the local machine. The hardware specification includes the NVIDIA GeForce RTX 5080 Laptop GPU with 16 GB of VRAM, which enables the execution of quantized models without any need for cloud services. Jupyter Notebook was used as the development environment, while Ollama provided the local inference server and REST API for the interaction with models. Several language models have been installed and tested, such as Qwen 3.5, Gemma 3, and Llama 3.1. Besides, the nomic-embed-text embedding model has been utilized for the purpose of performing semantic search within local documents.

TABLE 1: experimental environment

Item	Value
Operating system	Windows 11
Python	3.14.3 (pip 25.3), Jupyter Notebook with a Python 3 kernel
GPU	NVIDIA GeForce RTX 5080 Laptop GPU, 16 GB VRAM, driver 610.78, CUDA 13.3
Ollama	version 0.40.6, REST server at http://localhost:11434
Main model	qwen3.5 — 9.0B parameters, Q4_K_M, 6.55 GB on disk, vision and thinking capabilities
Second model	gemma3 — 3.35 GB on disk

Item	Value
Third model	llama3.1:8b — 8.0B parameters, Q4_K_M, 4.92 GB on disk
Embedding model	nomic-embed-text (274 MB), used for semantic retrieval

Exercise 1 was about verifying the environment installation

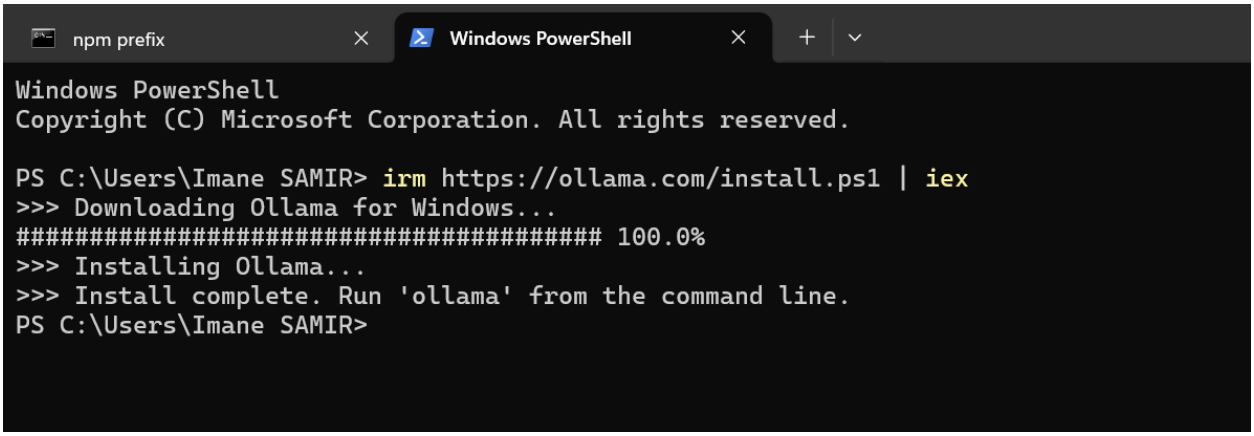


Fig 1: downloding Ollama localy



Fig 2: Environment check: Python version, Ollama 0.40.6 and server reachability

Ollama recognized the NVIDIA GPU automatically; there was no need for any CUDA configuration. Following the initial invocation, running ollama ps revealed that qwen3.5 had been loaded with 5.6 GB size, 100% GPU utilization, and context 4096 tokens.

```
print(ask("Say hello in one sentence.", think=False))
!ollama ps
```

```
[3.4s]
Hello! How can I assist you today?
NAME              ID              SIZE      PROCESSOR    CONTEXT    RUNNER      UNTIL
qwen3.5:latest    56671c2ab938    5.6 GB    100% GPU     4096       llamacpp    4 minutes from now
```

Fig 3: Response to a first prompt and `ollama ps`: the model runs 100% on the GPU.

III. Experiments:

1. First model and first prompts:

The qwen3.5 model was pulled by using ollama pull command (6.6 GB). The same subject, artificial intelligence, was prompted at three levels: the summary with five sentences, the explanation for a 10-year-old and the explanation for a computer scientist (27.8 s, 62.1 s and 68.3 s respectively with the default thinking mode). The three results obviously vary in vocabulary, content and structure: simple, then analogies like learning how to ride a bike, then technical terms like inductive bias and optimization landscapes.

Three notes should be kept. First, when we prompted the same topics twice, the results were different due to sampling. Second, although the answer for a computer scientist looked authoritative, it included inaccurate information (like the relationship between Transformers, attention and CNNs), hence, the local model can be authoritative yet incorrect. Third, the thinking mode slows down the responses significantly: a simple prompt required 3.4 s with think=False option.

2. Prompt Engineering:

In this phase we compared between two prompts regarding Directed Acyclic Graphs (DAGs). Both prompts gave the same role to the model (a university professor) and asked for same subject.

Table presents more details about both prompts:

TABLE II: prompt engineering with differents context and specificity

Element	Prompt A	Prompt B
Role	University professor	University professor (same)
Context	None given	First-year computer science student
Constraints	None	“Simple language”, one concrete example
Output format	Five numbered sections: definition, example, adjacency matrix, application, Python code	Not specified

Element	Prompt A	Prompt B
Specificity	Very specific about content	Specific about audience, free structure
Result	36.0 s; formal lecture, 3×3 adjacency matrix, networkx code	29.9 s; friendly tone, cake-baking analogy, no matrix or code

Only the role itself did not affect the content, so both answers remained those of a professor. While the audience (context) and restrictions affected the depth and tone, the specific output format was determined by Prompt A explicitly. The more explicit the instructions for the model are (what should include, to whom and in what form), the more predictable the output will be.

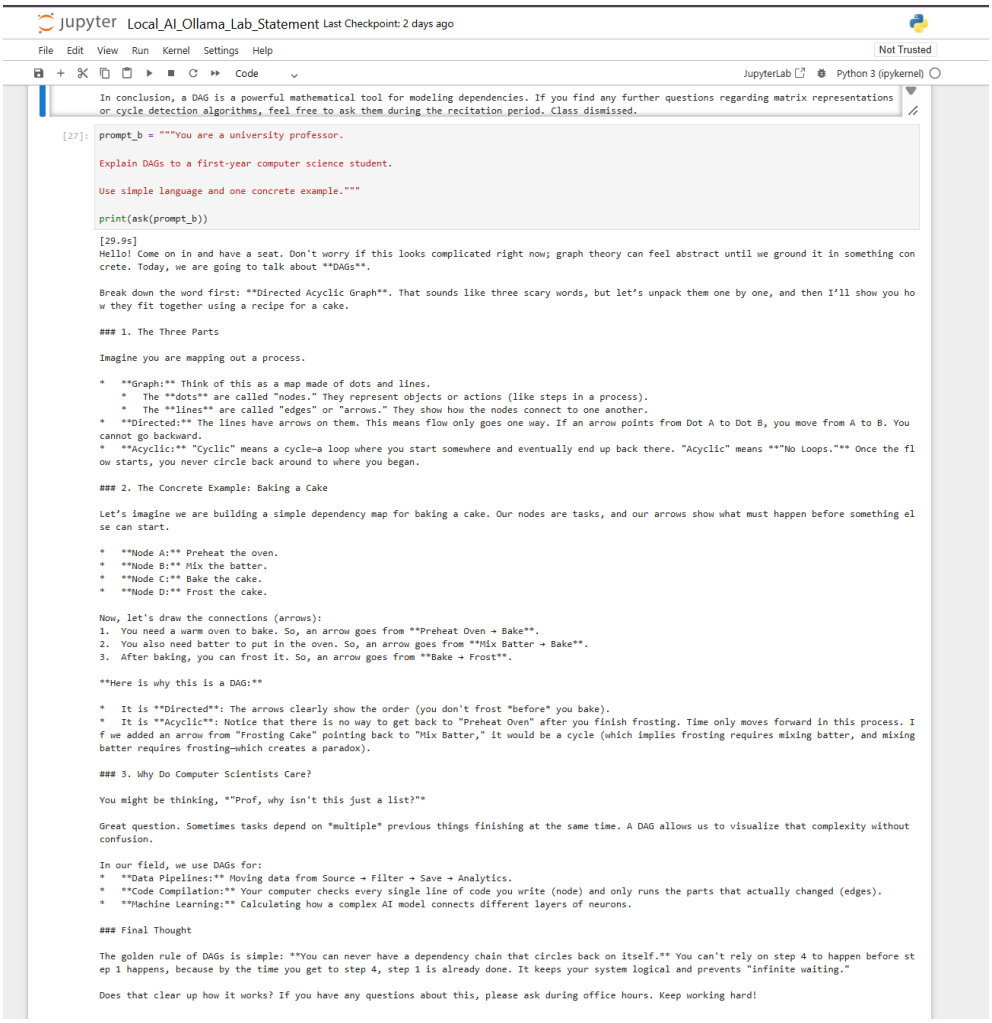


Fig 4: Prompt B: an answer adapted to a first-year student (29.9 s).

3. Model Management:

ollama list displays the models that downloaded to disk, while ollama ps displays the models that loaded into memory along with their size, processor, context, and the time when they will unload

(after about five minutes of inactivity). A downloaded model takes up disk space, while a loaded model takes up VRAM and responds instantly. ollama show provides technical specifications of qwen3.5.

Property	qwen3.5
Architecture / parameters	qwen35 / 9.0B
Quantization	Q4_K_M
Context length	262,144 tokens (4,096 used by default in the first requests)
Embedding length	4,096
Capabilities	completion, vision, tools, thinking (on by default)
License	Apache 2.0

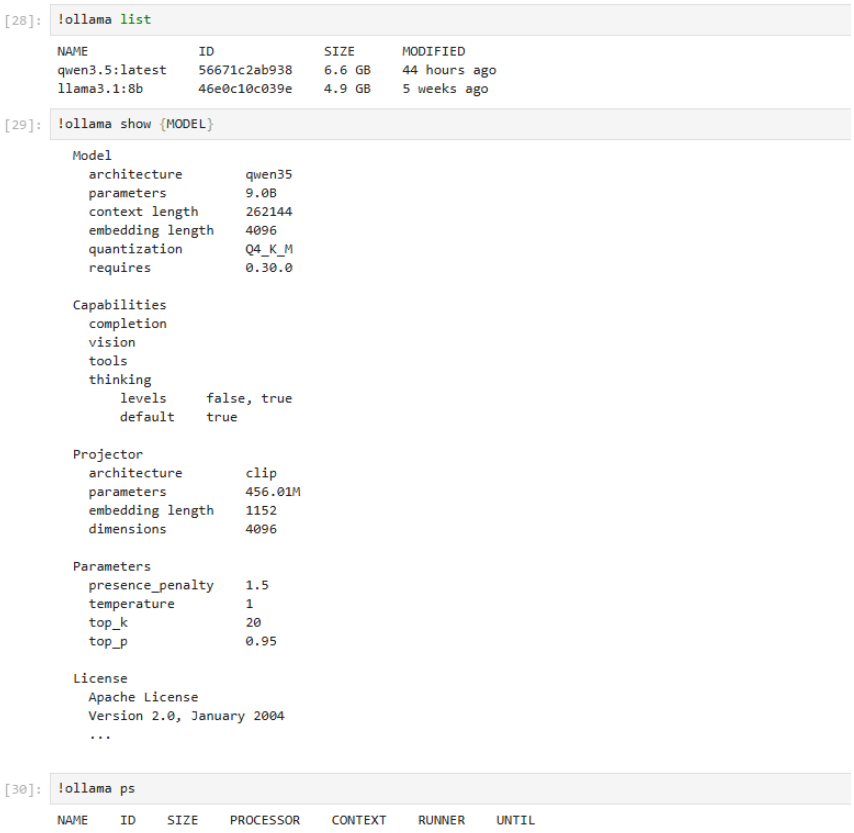


Fig 5: 'ollama list', 'ollama show' and 'ollama ps' (Exercise 6).

4. Comparing two Models on a Coding Task:

qwen3.5 and gemma3 asked to generate a documented Python function which checks for primality with the types, description, and time complexity. The functions generated compared later with the reference function on numbers from −5 to 1999.

Metric	qwen3.5	gemma3
Response time	16.3 s	20.2 s
Tokens generated	703	904
Speed	85.1 tokens/s	147.0 tokens/s
VRAM used	5.61 GB	2.88 GB
Wrong answers vs reference	none	none
Exceptions	none	7 (n = −5 to 1: raises ValueError)
Matches reference for n ≥ 2	yes	yes
Quality /5 (my evaluation)	4	4
Code correctness /5 (my evaluation)	5	4

Both models generate proper primality check logic. However, qwen3.5 utilizes a fast 6k±1 check and generates doctest examples, while its description claims that ValueError raised for $n < 2$ and returns False instead. gemma3 implements slower trial division, provides a more thorough description (including space complexity) and raises errors when $n \leq 1$, which is a matter of choice here. Both provide the correct time complexity description.

gemma3 generates tokens twice as fast and requires less memory. On the contrary, qwen3.5 finished sooner in this example (16.3 s against 20.2 s), because it generated less text, which is another reason tokens per second is a more relevant speed measure.

[39]:
```python
def is_prime_reference(n: int) -> bool:
    if n < 2: return False
    if n < 4: return True
    if n % 2 == 0: return False
    i = 3
    while i * i <= n:
        if n % i == 0: return False
        i += 2
    return True

#comparing:
def test(candidate, name):
    wrong, errors = [], []
    for n in range(-5, 2000):
        try:
            if candidate(n) != is_prime_reference(n):
                wrong.append(n)
        except Exception as e:
            errors.append(n)
    print(f"{name}:")
    print("  wrong answers :", wrong[:10] if wrong else "none")
    print("  exceptions    :", f"{len(errors)} (n = {errors[0]} to {errors[-1]})" if errors else "none")

    # fair check on valid inputs only (n >= 2)
    ok = all(candidate(n) == is_prime_reference(n) for n in range(2, 2000))
    print("  matches reference for n >= 2:", "✅ yes" if ok else "❌ no")

test(is_prime,  "qwen3.5 (is_prime)")
test(is_prime2, "gemma3 (is_prime2)")
assert all(is_prime(n) == is_prime_reference(n) for n in range(-5, 2000)), "Incorrect!"
print("✅ qwen3.5 matches the reference on -5..2000")
```

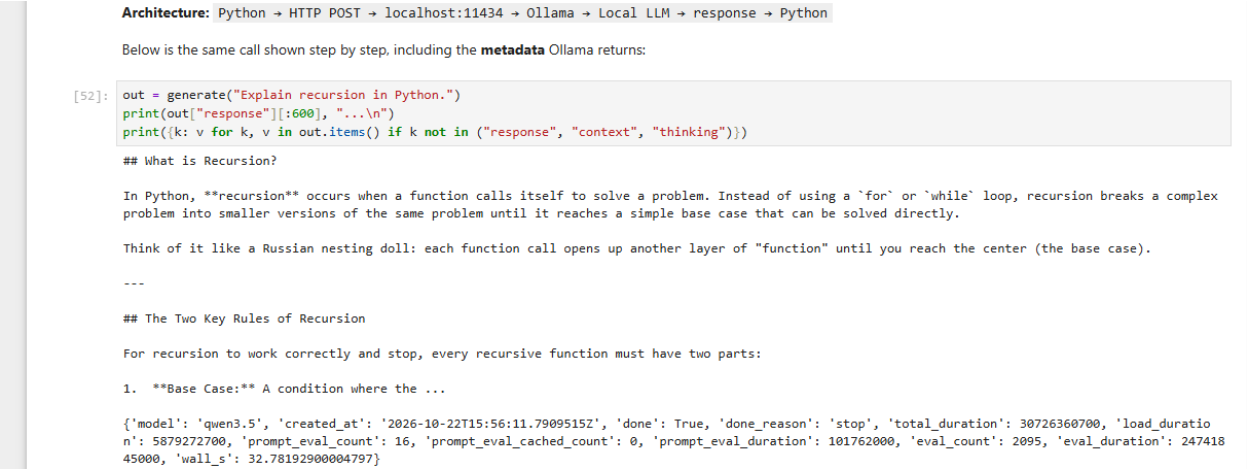
```
qwen3.5 (is_prime):
  wrong answers : none
  exceptions    : none
  matches reference for n >= 2: ✅ yes
gemma3 (is_prime2):
  wrong answers : none
  exceptions    : 7 (n = -5 to 1)
  matches reference for n >= 2: ✅ yes
✅ qwen3.5 matches the reference on -5..2000
```

Fig 6: Correctness test of the two generated functions against a reference implementation

5. Using Ollama from Python:

The minimum client in Python makes an HTTP POST to http://localhost:11434/api/generate with the model, the prompt and stream: false, and outputs the response field. Executing this as a different script on Windows initially resulted in a UnicodeEncodeError since the console encoding (cp1252) could not print the emoji generated by the model. This solved by executing it using py -X utf8. The same API call with metadata used to get timings per request (duration in nanoseconds).

The performance of the model estimated based on timing data from Ollama and the Python application. The total time indicated by Ollama was 30.7 seconds. It consisted of the time for loading the model of 5.9 seconds, which corresponds to loading the model into VRAM when it already unloaded. The process of prompt evaluation took 0.10 seconds to process 16 tokens. In the process of generation, the model produced 2,095 tokens in 24.7 seconds, which gives the approximate speed of generation of 85 tokens per second, including thinking tokens. The total wall-clock time estimated by the Python application was 32.8 seconds, which also includes HTTP communication overhead between the application and Ollama server.



Architecture: `Python → HTTP POST → localhost:11434 → Ollama → Local LLM → response → Python`

Below is the same call shown step by step, including the **metadata** Ollama returns:

```
[52]: out = generate("Explain recursion in Python.")
      print(out["response"][:600], "...\n")
      print({k: v for k, v in out.items() if k not in ("response", "context", "thinking")})
```

```
## What is Recursion?

In Python, **recursion** occurs when a function calls itself to solve a problem. Instead of using a `for` or `while` loop, recursion breaks a complex
problem into smaller versions of the same problem until it reaches a simple base case that can be solved directly.

Think of it like a Russian nesting doll: each function call opens up another layer of "function" until you reach the center (the base case).

---

## The Two Key Rules of Recursion

For recursion to work correctly and stop, every recursive function must have two parts:

1.  **Base Case:** A condition where the ...

{'model': 'qwen3.5', 'created_at': '2026-10-22T15:56:11.7909515Z', 'done': True, 'done_reason': 'stop', 'total_duration': 30726360700, 'load_duratio
n': 5879272700, 'prompt_eval_count': 16, 'prompt_eval_cached_count': 0, 'prompt_eval_duration': 101762000, 'eval_count': 2095, 'eval_duration': 247418
45000, 'wall_s': 32.78192900004797}
```

Fig 7: Answer and metadata returned by the Ollama API

6. Chatbot Memory:

The initial console of chatbot sends each question separately; thus, the bot is stateless: a follow-up question like “Give me an example of it” will sent to the model without any context. Even without any language instruction, qwen3.5 provided one question from English to Chinese; therefore, a system instruction required to control the output language.

Memory introduced through the history array and sending all questions with the conversation. In our test, the model received the instruction “My name is Ahmed.”, and the printed prompt for the second call included this sentence, followed by the answer from the model “Your name is Ahmed!”. No information stored by the model between the calls; the memory works in the application only. The /api/chat endpoint behaves identically when using structured messages with system/user/assistant roles; therefore, it gave the identical answer.

```
[61]: history.clear()
for q in ["My name is Ahmed.", "What is my name?"]:
    print("You:", q)
    print("AI :", ask_ai_memory(q), "\n")

You: My name is Ahmed.
AI : Nice to meet you, Ahmed! How can I help you today?

You: What is my name?
AI : Your name is Ahmed!

[62]: print("What the model actually received on the last call:\n" + "-"*50)
print("\n".join(history[:-1]) + "\nAssistant:")

What the model actually received on the last call:
--------------------------------------------------
User: My name is Ahmed.
Assistant: Nice to meet you, Ahmed! How can I help you today?
User: What is my name?
Assistant:
```

Fig 8: Memory test: the previous turns are part of the prompt sent to the model.

7. Local AI – Code Assistant

Two Java samples provided for review. For the Calculator division method (38.0 s), the model has correctly detected the ArithmeticException division by zero problem and recommended validation, documentation, and using double or BigDecimal instead. Nevertheless, the model's corrections included an extra method returning a boolean from a double one (which is not compilable), the Javadoc mentioned another exception than the one thrown by the sample, and it did not detect the true overflow case: Integer.MIN_VALUE/-1.

For the Student class (319.3 s, with thinking enabled on the shared GPU), it has detected all the expected two issues: the shadowed assignment age=age; (should be this.age=age;) and the fact that the age field is never initialized, and its final class compiles. Thus, a local model can consider a good first-pass reviewer which preserves the privacy of the source code but should have its corrections checked and compiled.

Asking for “ONLY JSON” in the prompt resulted in a neat JSON in my experiment (30.7 s) that parsed using a small extract_json() function. But nothing could guarantee that: a subsequent experiment may produce some text along with code fences or a different field name. Specifying a JSON Schema in the format parameter imposes restrictions on the generated output: the five fields always appeared, age was an integer; grade, a floating-point number; risk was either low, medium, or high.

Better: use Ollama's **structured outputs** — pass a JSON Schema in `format` so the output is constrained to be valid.

```
[68]: schema = {
          "type": "object",
          "properties": {
              "name": {"type": "string"},
              "age": {"type": "integer"},
              "grade": {"type": "number"},
              "risk": {"type": "string", "enum": ["low", "medium", "high"]},
              "recommendation": {"type": "string"},
          },
          "required": ["name", "age", "grade", "risk", "recommendation"],
      }

      out = generate(json_prompt, fmt=schema, think=False)
      student = json.loads(out["response"])
      print(json.dumps(student, indent=2))
      print("Type of grade:", type(student["grade"]))
```

```
{
  "name": "Ahmed",
  "age": 21,
  "grade": 15.5,
  "risk": "low",
  "recommendation": "Continue monitoring academic progress; minimal absences suggest strong attendance and engagement."
}
Type of grade: <class 'float'>
```

Fig 9: Schema-constrained output: typed fields and an enumerated risk level

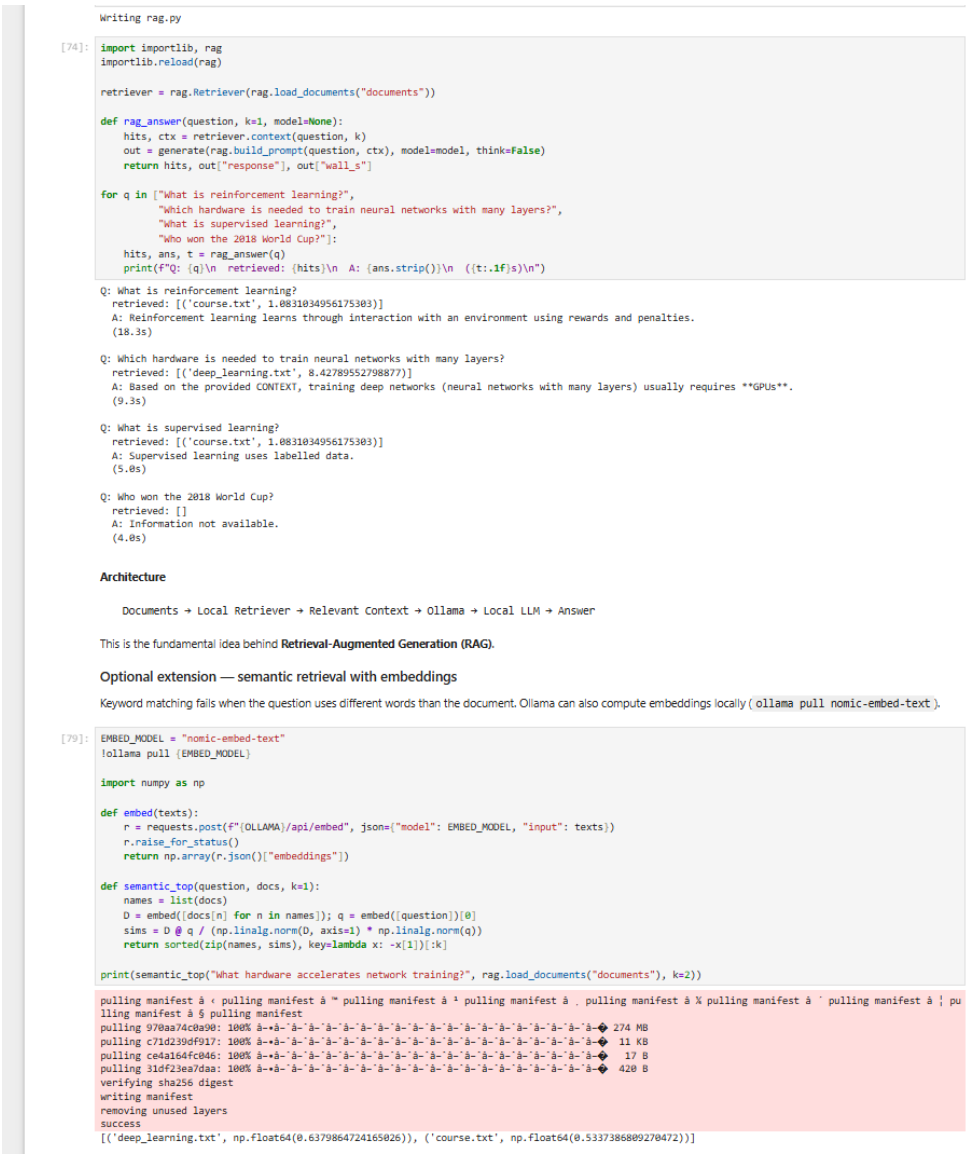
8. Document Assistant- Mini Rag:

The first assistant received the document with instructions “Say Information not available if the answer is not in the document” and the question about reinforcement learning was answered according to this document’s wording (17.8 s) while the question about quantum computing was answered with “Information not available.” (4.0 s)

This generalized into a mini-RAG system. The keyword (TF-IDF) retriever, which implemented in rag.py, extracts the most relevant document out of the course files (4 documents plus one remaining from the previous task) and only that document included in the prompt. Results of the first run:

Question	Retrieved (score)	Answer	Time
What is reinforcement learning?	course.txt (1.08)	Correct but short	18.3 s
Which hardware needed to train deep neural networks?	deep_learning.txt (8.43)	GPUs	9.3 s
What is supervised learning?	course.txt (1.08)	Correct but short	5.0 s
Who won the 2018 World Cup?	nothing	Information is not available.	4.0 s

There are two things to note here. First, the model was always following the grounding principle, including when the refusal for the out-of-scope question made. Second, the quality of retrieval influences the answer to some extent. This is since the keyword scoring method measures similarity in terms of words rather than meaning; hence, an incomplete answer obtained. The optional embedding model nomic-embed-text retrieves based on meaning: the question “What hardware accelerates network training?” answered first with respect to deep_learning.txt (similarity = 0.638 compared to the next one – 0.534).



```
Writing rag.py
```

```
[74]: import importlib, rag
importlib.reload(rag)

retriever = rag.Retriever(rag.load_documents("documents"))

def rag_answer(question, k=1, model=None):
    hits, ctx = retriever.context(question, k)
    out = generate(rag.build_prompt(question, ctx), model=model, think=False)
    return hits, out["response"], out["wall_s"]

for q in ["What is reinforcement learning?",
          "Which hardware is needed to train neural networks with many layers?",
          "What is supervised learning?",
          "Who won the 2018 World Cup?"]:
    hits, ans, t = rag_answer(q)
    print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties.
  (18.3s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.427895527988777)]
  A: Based on the provided CONTEXT, training deep networks (neural networks with many layers) usually requires **GPUs**.
  (9.3s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (5.0s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (4.0s)
```

Architecture

```
Documents → Local Retriever → Relevant Context → Ollama → Local LLM → Answer
```

This is the fundamental idea behind **Retrieval-Augmented Generation (RAG)**.

Optional extension — semantic retrieval with embeddings

Keyword matching fails when the question uses different words than the document. Ollama can also compute embeddings locally (`ollama pull nomic-embed-text`).

```
[79]: EMBED_MODEL = "nomic-embed-text"
!ollama pull {EMBED_MODEL}

import numpy as np

def embed(texts):
    r = requests.post(f"{OLLAMA}/api/embed", json={"model": EMBED_MODEL, "input": texts})
    r.raise_for_status()
    return np.array(r.json()["embeddings"])

def semantic_top(question, docs, k=1):
    names = list(docs)
    D = embed([docs[n] for n in names]); q = embed([question])[0]
    sims = D @ q / (np.linalg.norm(D, axis=1) * np.linalg.norm(q))
    return sorted(zip(names, sims), key=lambda x: -x[1])[:k]

print(semantic_top("What hardware accelerates network training?", rag.load_documents("documents"), k=2))
```

```
pulling manifest â ‹ pulling manifest â ™ pulling manifest â ¹ pulling manifest â ¸ pulling manifest â ¼ pulling manifest â ´ pulling manifest â ¦ pu
lling manifest â § pulling manifest
pulling 970aa74c0a90: 100% â–•â–ˆâ–â–â–â
```

[83]:
```
import base64

def analyze_image(path, model):
    with open(path, "rb") as f:
        b64 = base64.b64encode(f.read()).decode()
    prompt = """Analyze this image.

Identify:
1. Main objects
2. Environment
3. Text visible in the image
4. Possible interpretation"""
    return generate(prompt, model=model, images=[b64])

if os.path.exists(IMAGE_PATH):
    out = analyze_image(IMAGE_PATH, VISION_MODEL)
    print(out["response"], f"\n⏱ {out['wall_s']:.1f}s")
    from IPython.display import Image, display
    display(Image(IMAGE_PATH, width=400))
else:
    print(f"⚠️ Put an image named '{IMAGE_PATH}' next to this notebook, then re-run.")
```

```
Okay, here's an analysis of the image:

**1. Main Objects:**

*   **Two Kittens:** The primary focus is on two adorable, fluffy kittens. One is a pale orange tabby with white markings and a pink nose, and the oth
er is a gray tabby kitten with darker gray markings and a slightly darker nose.
*   **Circular Basket:** The kittens are sitting in a gray, circular wicker-style basket.

**2. Environment:**

*   **Indoor Setting:** The image is clearly taken indoors.
*   **Modern Furniture:** The background suggests a modern living room with a gray sofa and a gray cushion.
*   **Dark Background:** The background is intentionally dark, which helps to emphasize the kittens and the basket.

**3. Text Visible in the Image:**

*   There is no visible text within the image.

**4. Possible Interpretation:**

*   **Cuteness and Playfulness:** The image evokes feelings of cuteness, playfulness, and warmth. The kittens' inquisitive expressions (looking upward
s) suggest they are curious about something, possibly a person or something they see.
*   **Bonding and Companionship:**  The kittens sitting together in the basket conveys a sense of companionship and bond.
*   **Lifestyle/Home Decor:** The image also subtly communicates a lifestyle of pet ownership and potentially a stylish, modern home environment.

---

Would you like me to elaborate on any specific aspect of the image, such as the kittens' expressions or the overall mood it creates?
⏱ 70.6s
```



Fig 11: Local image analysis: objects, environment, text and interpretation

10. Local AI Web Application:

There is a Flask app (app.py and an HTML template) where a form is available; the question sent to Flask, which sends it to Ollama's /api/generate endpoint and returns the answer. The fact that thinking mode disabled (think: false) in the request results in answers returned in seconds rather than in a minute, and the question remains in the box on the page even after the answer shown. The disadvantages of this tool include lack of conversation memory, lack of streaming, and use of Flask's development server.



Fig 12: Local AI Assistant web page running at 127.0.0.1:5000.

11. Benchmark

Benchmarking script (benchmark.py) used to unload each model, give the same prompt (500-word description of reinforcement learning), and measure time to load, response time, number of tokens, speed, and memory.

Model	Disk (GB)	VRAM (GB)	Load (s)	Response (s)	Tokens	Tokens/s
qwen3.5	6.55	5.61	3.32	13.13	665	87.30
gemma3	3.35	2.88	3.69	11.29	826	152.16

Both models have loaded completely into VRAM (load size is equal to VRAM) in 3-4 seconds. Gemma3 works 1.7 times faster and consumes around half as much memory. System RAM usage (49.7% and 52.8%) is not much different, because both models operate in GPU memory.

```
    return float("nan"), float("nan")


def bench(model, prompt=PROMPT, runs=1):
    rows = []
    for _ in range(runs):
        unload(model)                    # cold start, so the load time is visible
        t0 = time.perf_counter()
        r = requests.post(f"{OLLAMA}/api/generate",
                          json={"model": model, "prompt": prompt, "stream": False,
                                "think": False}, timeout=900).json()
        wall = time.perf_counter() - t0
        total_gb, vram_gb = loaded_info(model)
        rows.append({
            "Model": model,
            "Disk size (GB)": next((m["size"] / 1e9 for m in requests.get(f"{OLLAMA}/api/tags").json()["models"]
                                    if m["name"].startswith(model.split(":")[0])), float("nan")),
            "Loaded size (GB)": total_gb,
            "VRAM (GB)": vram_gb,
            "Load time (s)": r.get("load_duration", 0) / 1e9,
            "Response time (s)": wall,
            "Tokens": r.get("eval_count"),
            "Tokens/s": r["eval_count"] / (r["eval_duration"] / 1e9),
            "System RAM used (%)": psutil.virtual_memory().percent,
        })
    return pd.DataFrame(rows)


if __name__ == "__main__":
    import sys
    print(pd.concat([bench(m) for m in sys.argv[1:]]).round(2).to_string(index=False))
```

```
Writing benchmark.py
```

[90]:
```
import benchmark; importlib.reload(benchmark)

bench_df = pd.concat([benchmark.bench(m) for m in (MODEL, MODEL2)], ignore_index=True)
bench_df["Quality /5"] = None   # <- fill in after reading both answers
bench_df.round(2)
```

[90]:

	Model	Disk size (GB)	Loaded size (GB)	VRAM (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	System RAM used (%)	Quality /5
0	qwen3.5	6.55	5.61	5.61	3.32	13.13	665	87.30	49.7	None
1	gemma3	3.35	2.88	2.88	3.69	11.29	826	152.16	52.8	None

[91]:
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))
bench_df.plot.bar(x="Model", y="Tokens/s", ax=ax[0], legend=False, title="Speed (tokens/s)", rot=0)
bench_df.plot.bar(x="Model", y="Loaded size (GB)", ax=ax[1], legend=False, title="Memory (GB)", rot=0)
bench_df.plot.bar(x="Model", y="Response time (s)", ax=ax[2], legend=False, title="Latency (s)", rot=0)
plt.tight_layout(); plt.show()
```

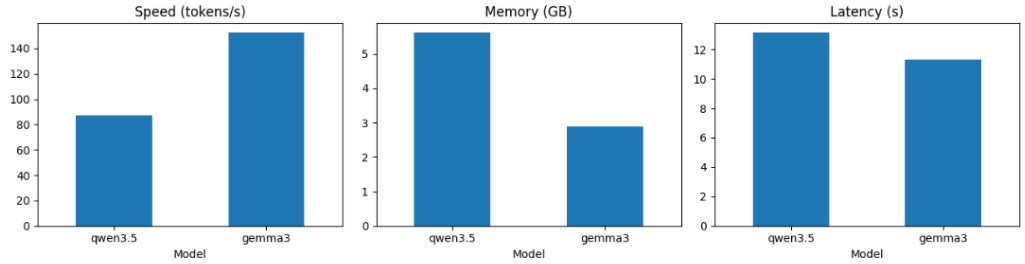


Fig 13: Benchmark table and charts: speed, memory and latency

IV. Final Project: FSTT Local AI Assistant

The final part combines all the above in one Flask web application. The application works completely local, uses the documents in the documents/ directory, maintains the conversation history of the last six messages, gives structured JSON for the chosen questions, and displays the answer, its sources, the response time, and the generation speed.

1. Architecture:

The implemented application operates within a local client-server structure that combines Flask, Ollama, and the local language model. The whole application implemented locally, Flask listens

to 127.0.0.1:5001, while Ollama serves as the inference server through localhost:11434, without using any external services. Users use an HTML page served by Flask to post questions related to the course. Flask takes care of the request and forwards it to Qwen 3.5 via Ollama's /api/chat endpoint. To answer questions based on local documents, Flask uses a TF-IDF retriever implemented in rag.py to search for two most relevant documents in the local documents/ folder. The system prompt ensures that the model confines its replies to the retrieved context. The application also keeps track of the history of the conversation since the last six user-assistant conversations are part of each request. To organize the output for structured replies, a JSON Schema is employed where the output categorized in fields like the topic, summary, important points, and difficulty. Lastly, the output answer, its source files, and performance metrics such as response time and tokens per second shown on the web application interface.

The overall architecture can summarize as follows:

Browser → Flask (app.py) → Conversation History and Retriever (rag.py) → Ollama /api/chat → Local LLM on the GPU → Answer, Sources, and Performance Metrics → Browser.

2. Results:

The application evaluated with four questions through its HTTP interface:

Question	Top sources (score)	Result	Time
What is reinforcement learning?	course.txt(1.08), reinforcement_learning.txt (1.02)	Correct definition (agent, state, action, rewards, policy)	11.76 s (includes model loading)
And how is it different from supervised learning?	course.txt (1.08), machine_learning.txt (1.06)	Correct contrast; “it” resolved through the history	3.78 s
Summarize deep learning (structured)	deep_learning.txt (2.43)	JSON with topic, summary, five key points, difficulty “hard”	10.37 s
What is quantum computing?	none	Information not available.	2.53 s

Test the final app from the notebook: launch it as a background process, send a few questions, then stop it.

[99]:
```
import subprocess, sys, time

env = {**os.environ, "MODEL": MODEL, "PORT": "5001"}
server = subprocess.Popen([sys.executable, "app.py"], cwd="LocalAI_Lab/source", env=env,
                          stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
time.sleep(3)

def app_ask(q, structured=False):
    r = requests.post("http://127.0.0.1:5001/ask", json={"question": q, "structured": structured}, timeout=600)
    return r.json()

try:
    for q, s in [("What is reinforcement learning?", False),
                 ("And how is it different from supervised learning?", False),   # tests history
                 ("Summarize deep learning", True),                              # structured
                 ("What is quantum computing?", False)]:                         # not in the docs
        j = app_ask(q, s)
        print(f"Q: {q}\nA: {j.get('structured') or j.get('answer') or j}\n"
              f"   sources={j.get('sources')} · {j.get('seconds')}s · {j.get('tokens_per_s')} tok/s\n")
finally:
    pass  # keep running so you can open http://127.0.0.1:5001 — stop in the next cell
```

```
Q: What is reinforcement learning?
A: Reinforcement learning is a type of machine learning where an agent interacts with an environment by observing a state, choosing an action, and rec
eiving rewards or penalties. The goal is to learn a policy that maximizes cumulative reward. Common algorithms include Q-learning and policy gradient
s.
   sources=[{'doc': 'course.txt', 'score': 1.08}, {'doc': 'reinforcement_learning.txt', 'score': 1.02}] · 11.76s · 23.3 tok/s

Q: And how is it different from supervised learning?
A: Reinforcement learning differs from supervised learning because it learns through interaction with an environment using rewards and penalties, wher
eas supervised learning uses labelled data to learn a mapping from inputs to outputs (classification, regression).
   sources=[{'doc': 'course.txt', 'score': 1.08}, {'doc': 'machine_learning.txt', 'score': 1.06}] · 3.78s · 27.8 tok/s

Q: Summarize deep learning
A: {'difficulty': 'hard', 'key_points': ['Uses multi-layer artificial neural networks', 'Learns from large amounts of data', 'CNNs for images', 'RNNs
and Transformers for sequences/language', 'Requires GPUs for training'], 'summary': 'Deep learning uses artificial neural networks with many layers to
learn representations from large amounts of data. It relies on convolutional neural networks for images, recurrent networks and transformers for seque
nces and language. Training deep networks usually requires GPUs.', 'topic': 'deep_learning.txt'}
   sources=[{'doc': 'deep_learning.txt', 'score': 2.43}, {'doc': 'course.txt', 'score': 0.21}] · 10.37s · 17.1 tok/s

Q: What is quantum computing?
A: Information not available.
   sources=[] · 2.53s · 41.1 tok/s
```

Fig 14: Test of the final application: history, structured output and out-of-scope question

In the web-based interface, “what is deep learning?” answered using deep_learning.txt in 5.32 s (45 tokens, 23.2 tokens/s). “Explain meta learning in 2 lines” resulted in the answer “Information not available.” in 3.63 s, and this is the right response because none of the documents contain information about meta-learning. This indicates that the assistant is as knowledgeable as its documents and that the knowledge base can expand simply by adding a text file.

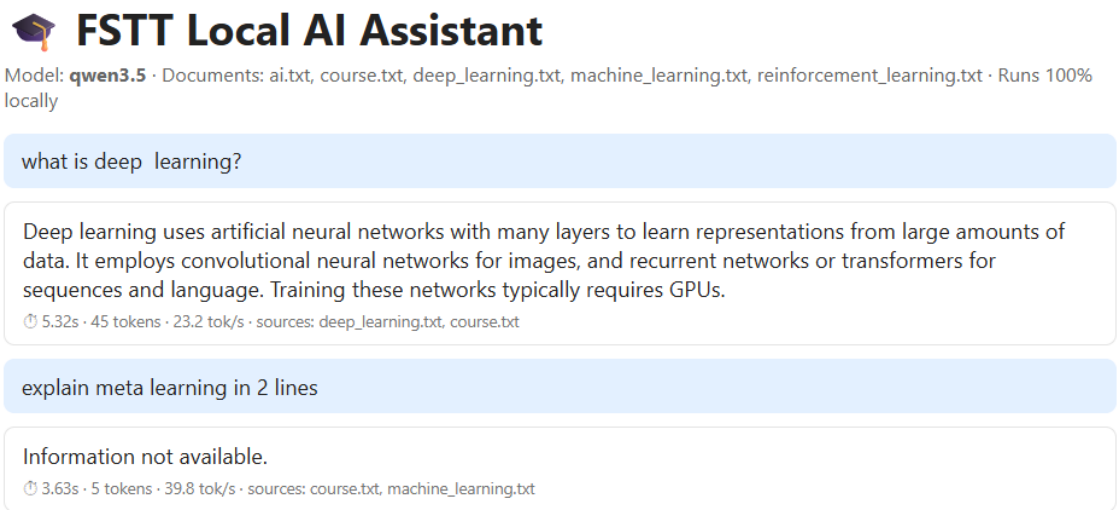


Fig 15: FSTT Local AI Assistant: answers with sources, time and tokens per second

Limitations and potential improvements. Keyword retriever will not work correctly on exceedingly small files (such as course.txt) and can replace them by the embedding retrieval method described in Section 3.9 with chunking of documents. History is global, and that will be fine for one-person demo but should use per-session storage otherwise. Streaming will make the system faster, and there is no authentication in the application, which is critical in case of public deployment.

V. Conclusion

This experiment demonstrated that a laptop with a 16 GB GPU can process useful language models fully locally, where Ollama manages the models and exposes a REST API interface to use them. From the metrics, we can see the main trade-off, as the larger model qwen3.5 takes twice the memory and runs slower compared to gemma3, with an improvement in quality being moderate. There are several useful insights from this experiment: thinking mode influences latency; the model does not have memory unless it is provided by the application; structured output with schema allows reliable integration; grounding in documents decreases hallucinations, although depending on the quality of retrieval; and generated code must be compiled and tested.

The last FSTT Local AI Assistant incorporates all these ideas into one web application that answers questions based on local documents, remembers the conversation, provides structured answers and its latency, without transmitting any data to the outside world. Some natural next steps would include embedding-based retrieval with document chunks, session history, streaming, and secured deployment in case of remote access.